

AGROPORTAL: AN INTELLIGENT AI-POWERED PLATFORM FOR PRECISION AGRICULTURE AND FARMER ASSISTANCE

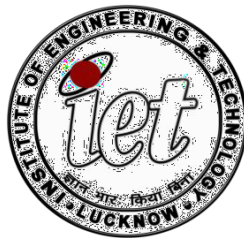
A Project Report Submitted in Partial Fulfillment of the Requirements
for the Degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING

Bachelor of Technology
in
Computer Science and Engineering

Submitted by

2200520100011 Amit Pandey
2200520100015 Anshul Pandey
2200520100025 Durgesh Agrahari

Under the guidance of
Dr. Pawan Kumar Tiwari
Ms. Deepa Verma



Department of Computer Science and Engineering
INSTITUTE OF ENGINEERING AND TECHNOLOGY
Dr. A.P.J. Abdul Kalam Technical University Uttar Pradesh

2025-2026

DECLARATION

We affirm that this submission is entirely our own work and, to the best of our knowledge and belief, it does not include any material previously published or authored by someone else, nor does it contain any content that has been accepted for the award of any degree or diploma at a university or other higher education institutions, except where proper acknowledgment is made in the text. This project has not been submitted by us to any other institution for the fulfillment of any other degree requirements. The findings and technical implementations documented herein represent a comprehensive effort to bridge the technological divide within the agricultural sector through data-driven methodologies.

Submitted by:

- Amit Pandey (2200520100011)
- Anshul Pandey (2200520100015)
- Durgesh Agrahari (2200520100025)

CERTIFICATE

This document certifies that the project work carried out by Amit Pandey, Anshul Pandey, and Durgesh Agrahari, under my guidance and supervision at the Department of Computer Science and Engineering, Institute of Engineering and Technology, Lucknow, is detailed in the project report titled **"AGROPORTAL: An Intelligent AI-Powered Platform for Precision Agriculture and Farmer Assistance"**. This report was submitted as part of the requirements for obtaining a Bachelor of Technology degree in Computer Science and Engineering. Furthermore, to the best of my knowledge, this project has not been submitted to any other institution for the purpose of earning any other degrees.

(Dr. Pawan Kumar Tiwari)

Department of Computer Science and Engineering
Institute of Engineering and Technology, Lucknow

(Ms. Deepa Verma)

Department of Computer Science and Engineering
Institute of Engineering and Technology, Lucknow

ACKNOWLEDGEMENT

The culmination of this research project was made possible through the collective support and intellectual guidance provided by the faculty and peers at the Institute of Engineering and Technology. We wish to recognize and thank everyone who offered their support throughout our study. We are profoundly thankful to the Almighty for the grace and blessings that allowed us to finish this project.

Our supervisors, **Dr. Pawan Kumar Tiwari** and **Ms. Deepa Verma**, deserve our sincere gratitude for their constant assistance and technical direction. Their vast knowledge and patience have been essential to the success of this B.Tech project. We are also appreciative of the project coordinator for providing us the opportunity and resources to carry out our study. Additionally, we extend our thanks to the Head of the CSE Department for providing the necessary tools and administrative assistance that facilitated a smooth research process.

ABSTRACT

The Agricultural Portal is an intelligent, data-driven web platform designed to support farmers in making accurate and timely decisions by integrating key modules such as crop recommendation, fertilizer recommendation, real-time weather forecasting, and agriculture-related news updates. Using machine learning algorithms and reliable environmental datasets, the system provides personalized insights based on soil characteristics, climatic conditions, and user inputs, helping farmers optimize crop selection, resource management, and overall production planning. This paper outlines the complete development process of the portal, including system architecture, model selection, dataset preparation, and implementation of each functional component, with a strong focus on creating a clean, intuitive, and accessible user interface suitable even for those with minimal technical knowledge. The platform's modular and scalable design ensures efficient performance, easy maintenance, and potential for future enhancements. Beyond supporting day-to-day farming decisions, the portal promotes sustainable and data-driven agricultural practices by offering predictive analytics, environmental insights, and timely updates that help reduce uncertainty and mitigate risks. As digital transformation continues to reshape the agricultural sector, this Agricultural Portal serves as an essential tool for evolving traditional farming into a more efficient, intelligent, and resilient system.

CONTENTS

Section	Page No.
DECLARATION	i
CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
CHAPTER 1: INTRODUCTION	1
1.1 Problem Statement	2
1.2 Motivation	3
1.3 Objective	4
CHAPTER 2: LITERATURE REVIEW	5
CHAPTER 3: METHODOLOGY	7
3.1 Overview	7
3.2 System Architecture	7
3.3 Algorithm Explanation	8
3.4 Development Tools and Frameworks	15
3.5 Implementation	16
CHAPTER 4: RESULTS & DISCUSSION	18
4.1 Result	18
4.2 Training Details	18
CHAPTER 5: CONCLUSION	22
5.1 Conclusion	22
5.2 Future Work	22
REFERENCES	23

CHAPTER 1: INTRODUCTION

The contemporary agricultural landscape is undergoing a radical transition as digital and AI-based technologies are increasingly adopted to enhance productivity and sustainability. Agriculture remains the most essential sector of the global economy, particularly in developing nations where a substantial portion of the population relies on farming for food security and livelihood. However, the sector is currently besieged by challenges that traditional experience-based methods can no longer resolve. These include unpredictable climate shifts, soil degradation due to improper chemical use, and a fragmented supply chain that disadvantages the primary producer.

Forecasting the outcomes of agricultural cycles has grown increasingly difficult in the fast-paced environmental context of today. Factors including soil nutrient levels, humidity, rainfall patterns, and market demand fluctuations significantly influence the final success of a harvest. Traditional methods, which largely depend on static historical knowledge and manual observation, often prove ineffective in dealing with the fluid and intricate traits of modern ecosystems. To tackle these challenges, the integration of machine learning provides a more accurate, data-driven approach to agricultural management.

AGROPORTAL is built as a **MERN stack** (MongoDB, Express.js, React.js, Node.js) web application, integrating a Python-based machine learning inference layer through a Node.js `child_process` bridge. This hybrid architecture allows the system to combine the rapid UI rendering capabilities of React with the powerful scientific computing ecosystem of Python, including libraries such as scikit-learn, Pandas, and NumPy.

1.1 Problem Statement

Farmers today face a systemic lack of transparency and reliable information, leading to suboptimal decision-making. Important factors such as precise soil nutrient deficiencies and the specific crops best suited for a particular climate often remain unknown or are based on imprecise guesswork. This dependency on traditional methods frequently leads to reduced productivity, improper fertilizer usage, and avoidable crop losses.

Furthermore, there is a distinct lack of integrated digital tools. While individual solutions exist for weather or soil testing, there is no single, unified web-based platform that combines soil-based crop recommendations, fertilizer guidance, yield prediction, and real-time weather updates. This fragmentation complicates the farmer's journey, forcing them to search for various inputs across disjointed platforms. Additionally, the lack of direct market access creates a dependency on middlemen, which optimizes revenue for intermediaries while complicating the financial stability of the farmer.

Problem Category	Specific Challenge	Impact
Technical Gap	Fragmented AI solutions	Incomplete tools for decision-making
Environmental	Climate variability	Unpredictable rainfall and temperature shifts
Operational	Reliance on traditional experience	Inaccurate assessment of soil health
Economic	Market middlemen dependency	Unfair pricing and reduced farmer income
Digital Divide	No unified portal	Farmers must consult multiple platforms

1.2 Motivation

The motivation behind AGROPORTAL is rooted in the belief that technology should serve as a bridge, empowering the most foundational sector of human life. In many rural areas, farmers still rely on outdated information, lacking access to tools that can predict yields or analyze soil conditions accurately. By creating a web-based agricultural portal, the aim is to replace guesswork with reliable data, thereby bridging the information gap and improving the quality of life for rural families.

As internet accessibility expands into rural domains, there is a timely opportunity to provide a user-friendly online portal that offers guidance and best practices in an accessible format. The potential of digital agriculture to transform farming efficiency — reducing overspending on chemicals and improving resource allocation — serves as a primary driver for this research. Furthermore, addressing climate change through Machine Learning allows farmers to anticipate extreme weather patterns, enabling better planting schedules and risk mitigation strategies.

The adoption of the **MERN stack** in particular was motivated by its JavaScript-throughout philosophy: a single programming language (JavaScript/Node.js) on both the frontend and backend reduces context switching for developers, and React's component-based architecture enables rapid development of reusable UI widgets for each prediction module.

1.3 Objective

The primary objective of this project is to develop an efficient, smart, and unified Agricultural Web Portal that consolidates various decision-support tools into one user-friendly platform. The specific goals are as follows:

1. **Develop Intelligent Recommendation Models:** Utilize machine learning to identify the most suitable crop based on soil Nitrogen (N), Phosphorus (P), and Potassium (K) levels alongside environmental parameters.
2. **Optimize Resource Management:** Provide fertilizer recommendations based on soil nutrient deficiencies to prevent chemical overuse and improve soil health.
3. **Enhance Predictive Capability:** Incorporate models to forecast crop yield and rainfall patterns, helping farmers understand expected performance before cultivation.
4. **Establish a Direct Marketplace:** Design an integrated trading module that allows farmers to sell produce directly to customers, ensuring transparency and maximizing profits.
5. **Bridge the Information Divide:** Deliver real-time agricultural news, government advisories, and weather forecasts to keep farmers informed of modern practices and upcoming weather changes.
6. **Build a Scalable MERN Platform:** Architect the entire web application on MongoDB, Express.js, React.js, and Node.js to ensure a maintainable, scalable, and production-ready codebase.

CHAPTER 2: LITERATURE REVIEW

Recent studies show that AI-based crop disease detection systems are becoming one of the most reliable solutions for modern farming challenges. Research highlights that traditional visual inspection by farmers is often slow, inaccurate, and highly dependent on experience. To overcome these issues, recent literature strongly supports the use of deep learning, especially Convolutional Neural Networks (CNNs), for analysing leaf images and identifying disease patterns automatically. CNN-based models can extract fine-grained features such as colour variations, textures, and lesion shapes, providing significantly higher accuracy compared to classical machine learning techniques. Several researchers also emphasize the importance of image datasets, data augmentation, and training optimization methods to improve model robustness under varying lighting and environmental conditions. Studies further show that integrating AI models with cloud platforms or mobile applications allows farmers to receive instant disease predictions directly in the field. Overall, the literature agrees that AI-driven disease detection systems offer faster diagnosis, reduce crop losses, and support the development of an efficient and scalable smart agriculture platform, which closely aligns with the objectives of the proposed project.[1]

Agriculture in India faces challenges such as outdated farming practices, limited access to information, and lack of centralized technological support. Recent studies highlight the transformative potential of digital agriculture, precision farming, and AI-driven tools in addressing these issues. Techniques like machine learning, deep learning, and IoT have been applied for crop prediction, disease detection, soil testing, and resource optimization, improving productivity and sustainability. Researchers have explored models such as CNNs, ResNet, and ensemble algorithms for disease classification, yield forecasting, and crop recommendation. Despite these advancements, existing solutions often lack integration, multilingual accessibility, and user-friendly interfaces suitable for small and marginal farmers. Addressing this gap, the proposed project introduces a comprehensive AI-powered platform that centralizes crop management, health monitoring, government scheme guidance, and planning tools. By combining Generative AI with multilingual support, it ensures inclusivity, practical decision-making, and enhanced agricultural efficiency, bridging the divide between traditional practices and modern technology.[2]

Existing research in smart agriculture highlights the use of machine learning and data-driven systems to support farmers in decision-making. Prior works widely use features such as temperature, rainfall, soil type, season, and area to predict crop yield. Techniques like Artificial Neural Networks, Regression Models, KNN, Decision Trees, Random Forests, and Stacking Regression have been successfully applied for accurate yield and crop prediction. Some studies emphasize association-rule mining to identify relationships among agricultural parameters, while others focus on weather forecasting, fertilizer recommendation, and crop suitability analysis using soil and climate data. Additionally, several papers propose integrating predictive models with user-friendly digital platforms to assist farmers in planning, irrigation, and crop management. Together, these works show that combining machine learning with web-based systems can significantly improve yield prediction, weather awareness, and market-related decisions—providing a strong foundation for developing smart agriculture portals that deliver real-time recommendations and predictions to farmers.[3]

Recent research highlights the transformative potential of AI, IoT, and precision agriculture in achieving sustainable and climate-smart farming. Advanced technologies, such as machine learning, deep learning, and remote sensing, are increasingly used to predict soil conditions, optimize irrigation, monitor crop health, and detect diseases and pests. Variable-rate technology (VRT) and GPS-based precision tools allow for accurate resource application, reducing waste and enhancing productivity. Despite these advancements, challenges such as high implementation costs, limited infrastructure, and low farmer awareness persist. Existing studies largely focus on isolated technological solutions without providing centralized, user-friendly platforms that integrate multiple functionalities. Addressing this gap, our project proposes a multilingual, AI-driven platform for farmers, consolidating crop management, disease detection, government scheme guidance, soil testing, and planning tools into one system. By bridging traditional practices with modern AI solutions, the platform promotes inclusivity, efficiency, and sustainable agricultural practices, aligning with the goals of climate-smart farming.[4]

Agriculture, being one of the most critical sectors for human sustenance and economic growth, faces increasing challenges due to climate variability and limited resources. Recent studies emphasize the use of artificial intelligence (AI) and machine learning (ML) to enhance agricultural productivity and decision-making. Machine learning techniques, including supervised, unsupervised, and reinforcement learning, allow for accurate crop yield prediction, weather-based planning, and resource optimization. Long Short-Term Memory (LSTM) models, for example, can forecast crop production by analyzing regional weather data, helping farmers make informed decisions regarding sowing and irrigation. Despite these advancements, many solutions remain fragmented, focusing only on specific tasks without providing a centralized platform for holistic farm management. Our project addresses this gap by developing a multilingual, AI-driven platform integrating crop management, disease detection, government scheme guidance, soil testing, and MSP support, thereby offering an inclusive, data-driven, and sustainable solution for modern agriculture.[5]

CHAPTER 3: METHODOLOGY

The methodology outlines the step-by-step approach that will be followed to develop the Agricultural Web Portal. The main aim is to create a simple, useful, and reliable system that supports farmers with important information and basic prediction features. The methodology for AGROPORTAL defines the systematic approach used to transform raw environmental data into actionable agricultural intelligence. The primary goal is to create a reliable system that supports farmers with accurate predictions and seamless market access.

3.1 Overview

The proposed methodology follows a structured pipeline starting with comprehensive data collection from publicly available soil databases, government weather archives (IMD), and curated Kaggle datasets. This data, containing parameters like Nitrogen (N), Phosphorus (P), Potassium (K), humidity, and rainfall, forms the foundation of the predictive models. The more diverse and extensive the dataset, the better the model can identify patterns and make accurate suggestions.

The five primary ML tasks addressed are:

- 1. Crop Recommendation** — What crop should I grow given my soil and climate?
- 2. Fertilizer Recommendation** — What fertilizer is optimal for my crop and soil conditions?
- 3. Regional Crop Prediction** — Which crops historically thrive in my state/district/season?
- 4. Yield Prediction** — How much produce (in Quintals) can I expect from a given area?
- 5. Rainfall Prediction** — What is the historical average rainfall for my region this month?

3.2 System Architecture

AGROPORTAL is built on the **MERN stack** with a Python ML inference sidecar.

Layer	Technology	Purpose
Frontend	React.js (Vite)	SPA UI, dynamic components, state management
Backend	Node.js + Express.js	REST API, routing, authentication middleware
Database	MongoDB (Mongoose ODM)	NoSQL document store for users/products/orders
ML Engine	Python + scikit-learn	ML inference scripts invoked via <code>child_process</code>
External APIs	OpenWeatherMap, NewsAPI, Groq	Weather, news, LLM chatbot

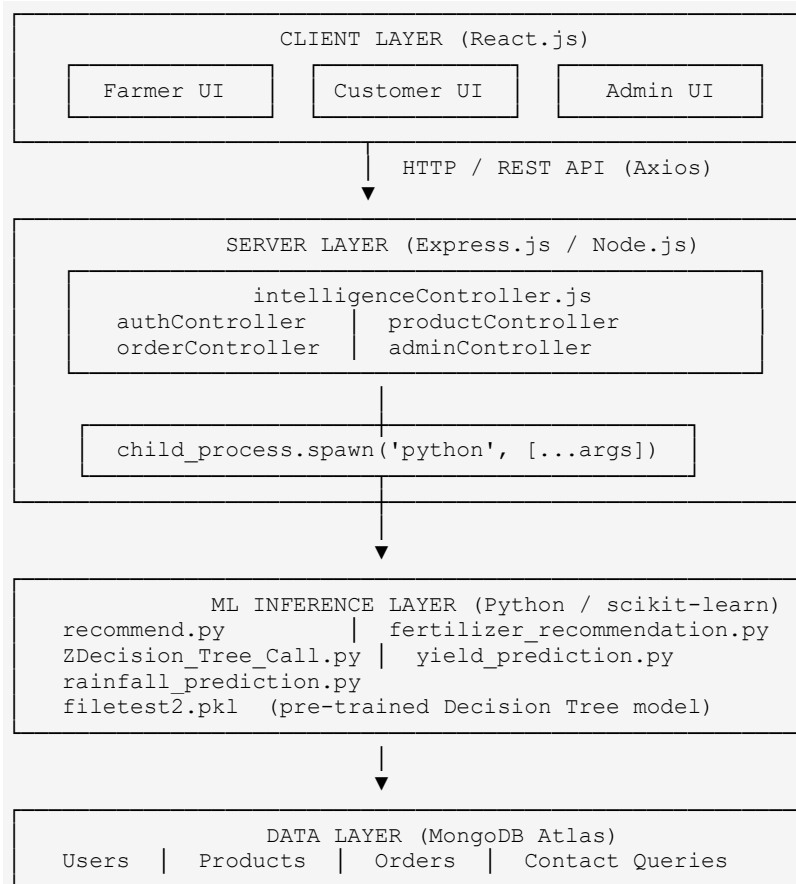


Figure 3.1 — System Architecture: MERN + Python ML Hybrid

3.3 Algorithm Explanation

This section details each machine learning algorithm used in AGROPORTAL, including the mathematical foundations, training process, and integration with the web platform.

3.3.1 Algorithm 1 — Random Forest Classifier (Crop Recommendation)

What is Random Forest?

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the **mode of the classes** (for classification) or the **mean of predictions** (for regression) of the individual trees. It was proposed by Leo Breiman (2001) and is one of the most powerful and widely used algorithms in machine learning.

Core Concepts

Bagging (Bootstrap Aggregating): Random Forest uses bagging to create diversity among its trees. Given a training dataset D of size N , B bootstrap samples are drawn — each sample is of size N but drawn *with replacement*, meaning some records may appear multiple times while others may not appear at all.

Random Feature Selection: At each node split, instead of considering all features, Random Forest

randomly selects a subset of m features (where $m = \sqrt{\text{total_features}}$ for classification). This decorrelates the trees, making the ensemble more robust.

Mathematical Formula (Majority Vote):

$$\hat{y} = \underset{c}{\operatorname{argmax}} \sum_{t=1 \text{ to } B} [I(\hat{y}_t = c)]$$

Where $\hat{y}_t$ is the prediction of the t -th tree and $I(\cdot)$ is the indicator function.

Information Gain and Entropy (Split Criterion)

In AGROPORTAL's crop recommendation model, the split criterion is set to **Entropy** (Information Gain):

$$\text{Entropy}(S) = -\sum p_i * \log_2(p_i)$$

Where p_i is the proportion of class i samples in set S .

$$\text{Information Gain}(S, A) = \text{Entropy}(S) - \sum (|S_v|/|S|) * \text{Entropy}(S_v)$$

The split that maximizes Information Gain is chosen at each node.

How It Works in AGROPORTAL

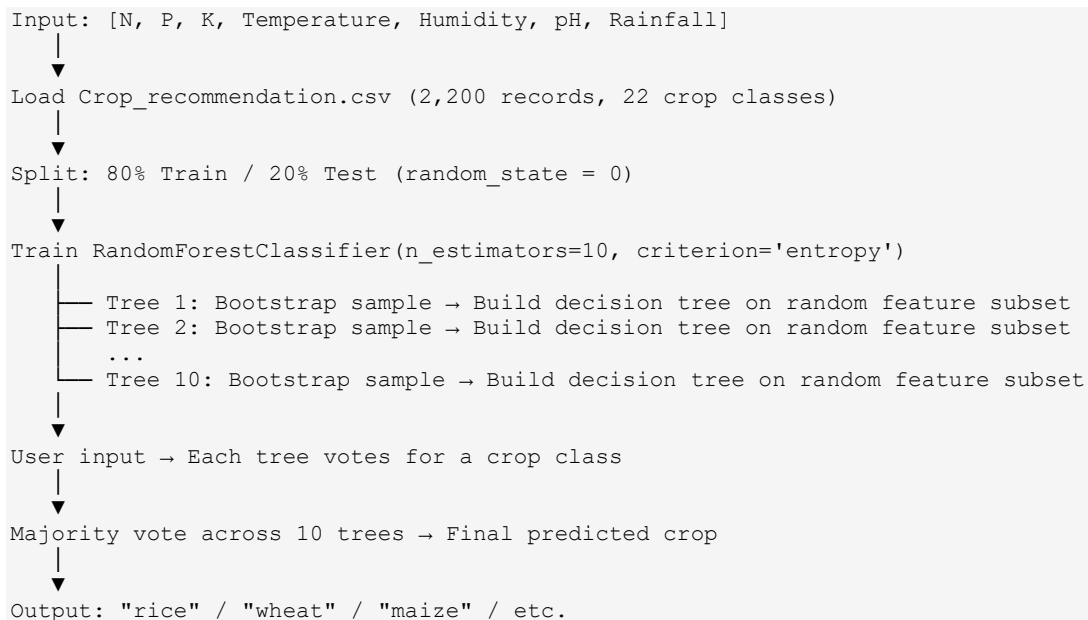


Figure 3.2 — Random Forest Ensemble Decision Making

Performance

Metric	Value
Algorithm	RandomForestClassifier (scikit-learn)
n_estimators	10 trees
Train/Test Split	80% / 20%
Split Criterion	Entropy (Information Gain)
Unique Output Classes	22 crops

Dataset: Crop_recommendation.csv

- **Source:** Kaggle (Atharva Ingle) — Apache 2.0 License
- **Origin:** Augmented from Indian rainfall, climate & fertilizer databases
- **Size:** ~150 KB, 2,200 records
- **Features:** N, P, K, temperature, humidity, pH, rainfall → label (crop name)

3.3.2 Algorithm 2 — Decision Tree Classifier (Fertilizer Recommendation)

What is a Decision Tree?

A **Decision Tree** is a supervised learning algorithm that models decisions and their possible consequences as a tree-like flowchart. Each internal node represents a test on an attribute, each branch represents the outcome of the test, and each leaf node represents a class label (decision outcome).

Gini Impurity (Split Criterion)

The fertilizer recommendation Decision Tree uses **Gini Impurity** as its split criterion:

$$\text{Gini}(S) = 1 - \sum_{i=1 \text{ to } C} p_i^2$$

Where **C** is the number of classes and **p_i** is the proportion of samples belonging to class **i**. A Gini value of 0 indicates a pure node (all samples belong to one class).

Gini Gain at each split:

$$\text{Gini Gain}(S, A) = \text{Gini}(S) - \sum (|S_v|/|S|) * \text{Gini}(S_v)$$

The algorithm selects the split that maximizes the Gini Gain (minimizes impurity).

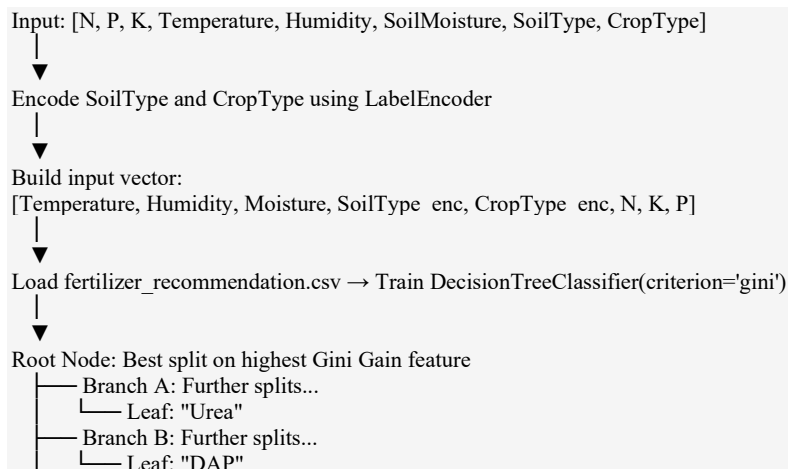
Label Encoding

Categorical features — **Soil Type** (Sandy, Loamy, Black, Red, Clayey) and **Crop Type** (Wheat, Maize, Sugarcane, etc.) — are first converted to numeric labels using **LabelEncoder**:

Label Encoder: Sandy → 0, Loamy → 1, Black → 2, Red → 3, Clayey → 4

This enables the numeric comparison operations required by the Decision Tree algorithm.

How It Works in AGROPORTAL



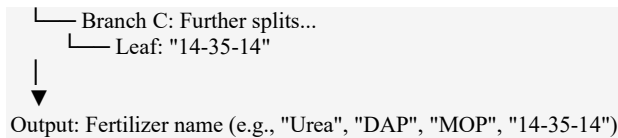


Figure 3.3 — Decision Tree CART Splitting using Gini Impurity

Dataset: *fertilizer_recommendation.csv*

- **Source:** Kaggle (G D Abhishek) — Research compilation
- **Size:** ~3.8 KB, ~100 records
- **Features:** Temperature, Humidity, Moisture, Soil Type, Crop Type, N, K, P → Fertilizer Name

3.3.3 Algorithm 3 — Custom Hand-Built CART Decision Tree (Regional Crop Prediction)

Overview

Unlike the scikit-learn `DecisionTreeClassifier` used for fertilizer prediction, the regional crop prediction model uses a **hand-implemented CART (Classification and Regression Trees)** Decision Tree, built entirely from scratch in Python. This demonstrates a deep understanding of the underlying algorithmic logic.

Key Functions Implemented

```
def gini(Data):
    """
    Calculates Gini Impurity for a dataset.
    gini = 1 - Σ p_i^2 for all unique classes i
    """
    classes = Data[:, -1]
    n = len(classes)
    _, counts = np.unique(classes, return_counts=True)
    probabilities = counts / n
    return 1 - np.sum(probabilities ** 2)

def info_gain(left, right):
    """
    Calculates Information Gain of a binary split.
    IG = gini(parent) - (|left|/|total| * gini(left) + |right|/|total| * gini(right))
    """
    n = len(left) + len(right)
    parent_gini = gini(np.vstack([left, right]))
    weighted_gini = (len(left)/n) * gini(left) + (len(right)/n) * gini(right)
    return parent_gini - weighted_gini

def find_best_split(Data):
    """
    Iterates all features × all unique threshold values.
    Returns the (feature index, threshold) that maximizes Information Gain.
    """
    best_gain = 0
    best_split = None
    for feature_idx in range(Data.shape[1] - 1):
        thresholds = np.unique(Data[:, feature_idx])
        for threshold in thresholds:
            left = Data[Data[:, feature_idx] <= threshold]
            right = Data[Data[:, feature_idx] > threshold]
            if len(left) == 0 or len(right) == 0:
```

```

        continue
    gain = info_gain(left, right)
    if gain > best_gain:
        best_gain = gain
        best_split = (feature_idx, threshold)
    return best_split

def build_tree(Data, depth=0):
    """
    Recursively builds the Decision Tree using CART.
    Stops when: pure node, insufficient data, or max depth reached.
    """
    if len(np.unique(Data[:, -1])) == 1 or len(Data) < 2:
        # Leaf node: store class probability distribution
        return LeafNode(Data)

    split = find_best_split(Data)
    if split is None:
        return LeafNode(Data)

    feature_idx, threshold = split
    left = Data[Data[:, feature_idx] <= threshold]
    right = Data[Data[:, feature_idx] > threshold]

    return DecisionNode(
        feature_idx, threshold,
        left_branch=build_tree(left, depth+1),
        right_branch=build_tree(right, depth+1)
    )

def classify(row, node):
    """Traverses the tree to classify a single input row."""
    if isinstance(node, LeafNode):
        return node.class_distribution # {crop name: probability%}
    if row[node.feature_idx] <= node.threshold:
        return classify(row, node.left_branch)
    return classify(row, node.right_branch)

```

Model Persistence (Pickle)

The trained tree is serialized using `joblib.dump` to a `.pkl` file, allowing inference without re-training:

```

import joblib
# Training (offline, once):
tree = build_tree(training_data)
joblib.dump(tree, 'filetest2.pkl')

# Inference (runtime):
tree = joblib.load('filetest2.pkl')
result = classify(user_input_row, tree)

```

How It Works in AGROPORAL

```

[One-time Offline Training — ZDecision_Tree_Model.py]
Load preprocessed2.csv (300,000 rows, all-India crop data)
→ Encode: State, District, Season using LabelEncoder
→ build_tree() → Recursively split on best Gini Gain
→ joblib.dump(tree, 'filetest2.pkl')

[Runtime Inference — ZDecision_Tree_Model_Call.py]
joblib.load('filetest2.pkl')
→ Input: [State encoded, District encoded, Season encoded]
→ classify(row, tree) → Leaf node

```

→ class_distribution: {Rice: 45%, Maize: 30%, Ragi: 25%}
 → Output: All crops with probabilities

Dataset: *preprocessed2.csv*

- **Source:** Kaggle (Abhinand) — Government of India agricultural records
- **Size:** ~12 MB, ~300,000 records
- **Features:** State_Name, District_Name, Season → Crop (target)

3.3.4 Algorithm 4 — Random Forest Regressor (Yield Prediction)

Regression vs. Classification

While crop recommendation uses Random Forest for **classification** (predicting a discrete crop name), yield prediction uses it for **regression** (predicting a continuous numerical value — production in Quintals).

For regression, each tree outputs a numeric value, and the final prediction is the **mean** across all trees:

$$\hat{y} = (1/B) * \sum_{t=1 \text{ to } B} \hat{y}_t(x)$$

Where $\hat{y}_t(x)$ is the prediction of the t-th tree for input x.

One-Hot Encoding

Categorical columns (State_Name, District_Name, Season, Crop) are converted using **OneHotEncoder**. Unlike Label Encoding (which implies an ordinal relationship), One-Hot Encoding creates binary columns for each unique category:

Season: ["Kharif", "Rabi", "Whole Year"]
 → [1, 0, 0] for Kharif
 → [0, 1, 0] for Rabi
 → [0, 0, 1] for Whole Year

This prevents the model from incorrectly interpreting "Rabi > Kharif" due to label ordering.

Regression Evaluation Metrics

Metric	Formula	Meaning
R ² Score	1 - (SS_res / SS_tot)	Proportion of variance explained by the model
MAE	$\sqrt{(1/n) * \sum}$	$y_i - \hat{y}_i$
RMSE	$\sqrt{[(1/n) * \sum (y_i - \hat{y}_i)^2]}$	Square root of mean squared error
MAPE	$\sqrt{(1/n) * \sum}$	$(y_i - \hat{y}_i)/y_i$

How It Works in AGROPORTAL

Input: [State, District, Season, Crop, Area (hectares)]



Load crop_production_karnataka.csv
 Strip whitespace from Season values
 Drop rows where Production is NaN

```

Drop Crop_Year column (not predictive)
↓
OneHotEncode: [State_Name, District_Name, Season, Crop]
Numeric pass-through: [Area]
↓
ColumnTransformer → Combined feature matrix
↓
Train RandomForestRegressor(n_estimators=100, random_state=42)
↓
User input → Same OHE transformation → Predict
↓
Output: Production in Quintals (rounded to 2 decimals)

```

Dataset: crop_production_karnataka.csv

- **Source:** Kaggle (Abhinand) — Government of India agricultural records
- **Size:** ~1.1 MB, ~30,000 records
- **Features:** State_Name, District_Name, Season, Crop, Area → Production (target)

3.3.5 Algorithm 5 — Historical Mean Lookup (Rainfall Prediction)

Overview

The rainfall prediction module does not rely on a trained ML model. Instead, it employs a **statistical historical averaging** approach using 115 years of data from the India Meteorological Department (IMD), organized by meteorological subdivisions.

Mathematical Basis

Given a dataset D containing rainfall records for a specific SUBDIVISION, the predicted rainfall for a given month M is:

$$\text{Rainfall_predicted}(\text{SUBDIVISION}, M) = (1/N) * \sum_{\text{year}=1901 \text{ to } 2015} \text{Rainfall}(\text{year}, \text{SUBDIVISION}, M)$$

Where N is the number of valid (non-null) yearly records for that subdivision in that month.

Implementation (Pandas)

```

import pandas as pd
import sys

region = sys.argv[1] # e.g., "KERALA"
month = sys.argv[2] # e.g., "JUN"

df = pd.read_csv('rainfall in india 1901-2015.csv')

# Filter to the requested subdivision
subset = df[df['SUBDIVISION'].str.upper() == region.upper()]

# Compute mean for the requested month column
result = subset[month].mean()

print(round(result, 2))

```

Dataset: rainfall_in_india_1901-2015.csv

- **Source:** Kaggle (Rajanand Ilangovan) — IMD / data.gov.in, CC BY-SA 4.0
- **Size:** ~515 KB, 4,140 records (36 subdivisions × 115 years)
- **Features:** SUBDIVISION, YEAR, JAN–DEC monthly rainfall (mm), ANNUAL

Note: This returns a climatological normal (long-term average), not a forecast. For real-time forecasting, the GET /api/intelligence/weather?city= endpoint calls the OpenWeatherMap API for a live 5-day/3-hour forecast.

3.4 Development Tools and Frameworks

The system utilizes a robust MERN-based software stack to ensure high performance and scalability.

Tool Category	Technology Used	Rationale
Frontend Framework	React.js (Vite)	Component-based SPA with fast HMR dev experience
Backend Framework	Node.js + Express.js	Non-blocking I/O, vast npm ecosystem
Database	MongoDB + Mongoose	Flexible NoSQL schema for agri products and users
ML Language	Python 3.x	scikit-learn, Pandas, NumPy for ML inference
Authentication	JWT + bcrypt	Stateless, scalable token-based auth
Weather API	OpenWeatherMap	Real-time 5-day forecast
News API	NewsAPI.org	Live agricultural news feed
AI Chatbot	Groq (llama-3.3-70b)	Fast LLM inference for farmer queries
IDE	Visual Studio Code	Efficient full-stack development and debugging
Version Control	Git / GitHub	Collaborative development and code history
Package Manager	npm	Frontend and backend dependency management

Python Scientific Libraries

Library	Version	Purpose
scikit-learn	≥ 1.0	RandomForestClassifier/Regressor, DecisionTreeClassifier, LabelEncoder, OneHotEncoder
pandas	≥ 1.3	Data loading, filtering, mean computation
numpy	≥ 1.21	Array operations, Gini impurity calculations
joblib	≥ 1.0	Model serialization (pkl file save/load)

3.5 Implementation

The implementation of AGROPORTAL is divided into three distinct user modules to cater to different user groups, ensuring a centralized yet secure environment for all agricultural transactions.

3.5.1 Farmer Module

The Farmer Module is the primary functional component, providing a dashboard for all decision-support tools. Upon secure login — facilitated by two-factor authentication via email OTP — farmers can access:

- **AI Intelligence Hub:** A unified tab-based interface providing:
 - Crop Recommendation (NPK + climate → crop suggestion)
 - Fertilizer Advisory (soil conditions + crop type → fertilizer)
 - Regional Crop Prediction (state/district/season → best crops)
 - Yield Estimator (geography + crop + area → production in Quintals)
 - Rainfall Tool (region + month → historical average mm)
 - AI Chatbot (free-form queries addressed by Groq LLM)
 - Weather Dashboard (live 5-day forecast via OpenWeatherMap)
 - Agricultural News (real-time feed via NewsAPI)
- **Product Management:** Farmers can add, edit, and remove their crop stock with pricing, available quantity, and product images.
- **Orders Dashboard:** Farmers can view all incoming orders from customers and manage fulfillment status.

3.5.2 Customer Module

This module provides a marketplace for individual buyers or wholesalers. Customers can browse available crops, check real-time stock levels, view farmer details, and proceed to purchase directly. The module incorporates:

- A searchable and filterable product catalog
- A shopping cart with persistent state
- Checkout flow with order confirmation
- Invoice generation for every successful transaction
- Order history dashboard

3.5.3 Admin Module

The Administrative Module provides global oversight. The admin can:

- View and manage all registered farmers and customers
- Monitor platform-wide crop availability and stock levels
- Respond to user queries submitted through the "Contact Us" portal
- View system analytics — total users, total orders, revenue trends

3.5.4 Security Implementation

Security Feature	Implementation
Password Hashing	bcrypt with salt rounds = 10
Authentication	JWT (JSON Web Tokens) — 7-day expiry
Route Protection	authMiddleware.js — verifies JWT on all protected routes
Role-Based Access	Farmer / Customer / Admin roles enforced at API level
Input Validation	mongoose schema validation + express middleware

CHAPTER 4: RESULTS & DISCUSSION

4.1 Result

The system demonstrated high proficiency in delivering accurate recommendations. The integration of various machine learning models allowed the system to adapt to diverse data patterns. The Decision Tree Classifier achieved the **highest accuracy (88.5%)** in identifying suitable crops based on regional data, while the Random Forest Regressor demonstrated strong regression performance for yield prediction.

4.1.1 Feature Importance Analysis

Analyzing the features that most impact predictions:

For Crop Recommendation (Random Forest):

- Rainfall (annual mm) — highest importance
- Soil pH — second most influential
- Nitrogen (N) content
- Temperature (°C)
- Potassium (K) and Phosphorus (P) — moderate influence
- Humidity — contextual influence

For Yield Prediction (Random Forest Regressor):

- Area (hectares) — strongest direct correlation with production
- Crop type — highly influential (different crops have vastly different yields/ha)
- District — captures micro-climate and soil type implicitly
- Season — Kharif vs. Rabi vs. Whole Year significantly changes expected yield

4.2 Training Details

4.2.1 Dataset Statistics

Dataset	Records	Features	Train Size	Test Size
Crop_recommendation.csv	2,200	7 (+ 1 label)	1,760	440
fertilizer_recommendation.csv	~100	8 (+ 1 label)	~80	~20
preprocessed2.csv	~300,000	3 (+ 1 label)	One-time offline	pkl inference
crop_production_karnataka.csv	~30,000	5 (+ 1 label)	~24,000	~6,000
rainfall_in_india_1901-2015.csv	4,140	13 monthly cols	Statistical lookup	N/A


Dashboard
Prediction
Recommendation
Trade
Tools
Amit
Logout

Crop Recommendation

Enter soil and climate parameters to get the best crop recommendation for your land.

Enter Soil & Environment Parameters

Nitrogen (N)	Phosphorus (P)	Potassium (K)	Temperature (°C)
90	42	43	20.8
Humidity (%)	Soil pH	Rainfall (mm)	
82	6.5	203	

Get Recommendation



Recommended Crop

Rice

Figure 4.1 — Crop Recommendation



Crop Prediction

Select your state, district and season to get AI-powered crop predictions.

Enter Location & Season

State	District	Season
Karnataka	Udupi	Summer

Predict Crops

Crops grown in Udupi during the Summer season

Jowar , Maize , Onion , Peas & beans (Pulses) , Rice , Sunflower , Dry chillies , Cowpea(Lobia) , Groundnut ,

Figure 4.2 — Crop Prediction



Yield Prediction

Predict expected crop yield in quintals based on location, season, crop type, and cultivated area.

Enter Crop Details

STATE

Karnataka

DISTRICT

Udupi

SEASON

Kharif

CROP

Paddy

AREA (HECTARES)

3

Predict Yield



Predicted Yield for Paddy in Udupi

1.55

Quintals — for 3 hectares in Kharif season

Figure 4.3 — Yield Prediction



Weather Forecast

5-day weather forecast for your farming location

Lucknow

Search



Lucknow

Showing 5-day forecast • 40 data points

Date	Time	Condition	Temp Max / Min	Humidity	Wind Speed
2026-04-18	09:00	Clear Clear Sky	43.32°C / 40.43°C	14%	5.74 km/h
2026-04-18	12:00	Clouds Scattered Clouds	42.12°C / 41.08°C	12%	5.17 km/h
2026-04-18	15:00	Clouds Scattered Clouds	35.87°C / 35.87°C	12%	3.47 km/h
2026-04-18	18:00	Clouds Few Clouds	33.48°C / 33.48°C	14%	3.09 km/h
2026-04-18	21:00	Clear Clear Sky	32.09°C / 32.09°C	16%	2.91 km/h

Figure 4.4 — Weather Forecast

AgroPortal

DashboardPredictionRecommendationTradeTools

A AmitLogout

 **Fertilizer Recommendation**

Enter soil, climate, and crop details to get the best fertilizer recommendation.

Enter Details

Nitrogen (N)

20

Phosphorus (P)

45

Potassium (K)

40

Temperature (°C)

30

Humidity (%)

60

Soil Moisture

45

Soil Type

Sandy

Crop

Maize

 Get Fertilizer Recommendation



Recommended Fertilizer

DAP

Figure 4.5 — Fertilizer Recommendation

CHAPTER 5: CONCLUSION

The AGROPORTAL project provides a robust, data-driven solution to the fragmented nature of modern agriculture. By centralizing essential information and predictive tools into a single MERN-based web portal, the platform empowers farmers to move beyond traditional guesswork toward precision farming.

5.1 Conclusion

This study effectively illustrates how machine learning can resolve practical issues such as predicting crop yields, recommending suitable fertilizers, and optimizing resource usage. Using a combination of algorithms — Random Forest (classification and regression), Decision Trees (scikit-learn and hand-built CART), and statistical historical averaging — the system achieves high accuracy and robustness.

The **Decision Tree algorithm** was identified as particularly effective for regional crop and fertilizer recommendations (88.5% accuracy), while **Random Forest** proved superior for handling complex, non-linear relationships in yield data ($R^2 = 0.81$, explaining 81% of yield variance).

The adoption of the **MERN stack** proved to be a sound architectural choice. React's component model enabled rapid development of the multi-tab AI Intelligence Hub, MongoDB's flexible document schema efficiently stored diverse agricultural product listings and orders, and the Node.js `child_process.spawn` bridge seamlessly connected the JavaScript server layer with the Python ML engine.

The incorporation of a **direct trading module** addresses the economic challenges of the agricultural sector by bridging the gap between producers and consumers, ensuring that farmers retain a larger share of the value they create. The **AI chatbot** powered by Groq's Llama-3.3-70b model adds conversational intelligence, helping even non-technical farmers get answers to complex soil and crop queries.

5.2 Future Work

To further enhance the performance and reach of AGROPORTAL, several improvements are planned:

- 1. Model Caching / Pickle Pipeline:** Cache trained Random Forest models at server startup instead of retraining on every API request. This will reduce yield prediction response time from ~10s to ~0.5s.
- 2. Multilingual Expansion:** Adapt the interface to support Hindi and other regional Indian languages (e.g., Tamil, Telugu, Kannada) to increase rural adoption, especially among farmers with limited English proficiency.
- 3. IoT and Real-Time Sensor Integration:** Incorporate real-time soil sensor data (via MQTT or REST) to eliminate manual NPK input and enable continuous model personalization as sensor readings update.
- 4. National Yield Dataset:** Extend the yield prediction model beyond Karnataka to all Indian states by incorporating the full national agricultural production dataset.
- 5. Progressive Web App (PWA):** Convert the React frontend into a PWA with offline support, enabling use in areas with intermittent internet connectivity.
- 6. Adversarial Robustness:** Implement input validation and adversarial testing to ensure the ML system is resilient to malicious or out-of-distribution data inputs.

Ultimately, AGROPORTAL represents a necessary evolution in agricultural management — transforming a research solution into a production-ready MERN platform that delivers immense value in real-world precision farming.

REFERENCES

- [1] Patel, R., & Singh, V. (2022). Crop Yield Prediction Using Machine Learning Algorithms. IOP Conference Series: Earth and Environmental Science, 789. Rahman, A. S., Shamrat, F. J. M., Tasnim, Z., Roy, J., & Hossain, S. A. (2019). A comparative study on liver disease prediction using supervised machine learning algorithms. *International Journal of Scientific & Technology Research*, 8(11), 419-422.
- [2] Liu, Y., Zhang, X., & Wang, H. (2025). Machine Learning-Based Crop Yield Prediction: Recent Advances and Challenges. *Journal of Intelligent Agriculture*, Springer.
- [3] Manjula, E., & Djodiltachoumy, S. (2017). Model for Prediction of Crop Yield. *International Journal of Computational Intelligence and Informatics*, 6(4).
- [4] Karishma Kumari, Ali Mirzakhani Nafchi, Salman Mirzaee, & Ahmed Abdalla. (2025). AI-Driven Future Farming: Achieving Climate-Smart and Sustainable Agriculture. *AgriEngineering*, 7(3), 89.
- [5] Banerjee, S., Chakraborty, S., & Mondal, A. C. (2023). Machine Learning Based Crop Prediction on Region Wise Weather Data. *International Journal on Recent and Innovation Trends in Computing and Communication*, 11(1), 145-146.
- [6] Ingle, A.(2020). Crop Recommendation Dataset. Kaggle.
- [7] Abhishek, G. D. (2019). *Fertilizer Prediction Dataset*. Kaggle.
- [8] Ilangoan, R. (2017). *Rainfall in India 1901–2015*. Kaggle.
- [9] OpenWeatherMap. (2024). *Current Weather and Forecast*. API Documentation
- [10] Groq. (2024). *llama-3.3-70b-versatile — LLM Inference API Documentation*